

UATimer: Lightweight Context-Aware Power Management for Heterogeneous IoT Systems

Geunsik Lim

Abstract—Energy-constrained Internet-of-Things (IoT) devices must enter low-power states aggressively without imposing unacceptable wake-up delay. Fixed inactivity timeouts are inexpensive but insensitive to changing activity, while learning-based policies add training, inference, and model-management overhead. This paper presents UATimer, a portable power-management policy layer that combines the classical exponential-average idle estimator with bounded, context-selected energy–latency operating modes. A break-even floor prevents unprofitable transitions, and a latency-critical mode can widen the threshold or inhibit suspension while an interactive session is active. The policy is event-driven, requires fewer than 500 lines of code and less than 1 KB of policy state, needs no training phase, and on Linux and Android runs in user space without kernel modification. We evaluate the mechanism on a smartphone, a tablet, and an IoT sensor node using web browsing, video playback, on-device AI inference, AR/VR gaming, and periodic sensing workloads. Across the reported workload means, UATimer reduces energy by 20.5–37.5% relative to the default baseline and by 7.9–16.7% relative to a fixed-timeout policy while keeping mean event delay below 50 ms. In the AR/VR workload, it reports a 0.5% frame-drop rate and 1.2 ms jitter. Against a lightweight linear-regression predictor, its mean energy is within 4% without an offline training phase. The results position UATimer as an inspectable policy layer between static timeouts and heavier predictive power managers, and we identify the measurements still required to establish its generality.

Index Terms—Internet of Things, constrained devices, dynamic power management, energy efficiency, adaptive timeout, context-aware systems, embedded systems.

I. INTRODUCTION

BATTERY-operated IoT and mobile-edge platforms increasingly execute interactive, sensing, communication, and inference workloads on the same device. Their energy budget is constrained not only by computation but also by repeated transitions among active, idle, and low-power states. A timeout that is too long wastes energy during idle intervals, whereas an overly aggressive timeout increases wake-up frequency and response delay. This tension is particularly important for systems that mix latency-sensitive tasks with long background idle periods.

The question of *when* to shut down an idle component is the classical dynamic power management (DPM) problem [1]. Fixed timeouts, predictive shutdown based on idle-interval history [2], [3], adaptive timeouts [4], [5], and stochastic or event-driven policies [6] have been studied extensively, and modern operating systems embed history-based idle governors [7]. Network-assisted duty cycling and IoT power-saving modes

further reduce active time, but their energy–latency behavior depends strongly on radio and paging configuration [8], [9]. Mobile-system studies likewise show that idle and suspended states materially affect device energy use [10].

Recent work has explored context-aware mobile scheduling [11], predictive sleep scheduling [9], and learning-based sleep policies for energy-constrained IoT systems [12], [13]. For small embedded platforms, however, even lightweight models introduce additional state, inference logic, and model-management complexity. At the other extreme, OS idle governors are tied to a specific kernel and processor idle-state interface and do not receive application-level context. This leaves a practical gap for a policy that (i) reuses a well-understood, training-free estimator, (ii) accepts coarse application context with bounded and inspectable effect, and (iii) can be carried across Linux, Android, and microcontroller firmware with only the suspend primitive changed.

This paper addresses that gap with UATimer, a *user-aware adaptive timer*. We do not claim a new idle-interval predictor: the exponential average of past idle intervals that UATimer uses is the estimator of classical predictive shutdown [3]. The contribution lies in how the estimator is bounded, coupled to context, and deployed across heterogeneous endpoints.

The main contributions are as follows:

- We design a portable policy layer that separates an online timeout policy from the platform suspend/resume mechanism. On Linux and Android it runs in user space without kernel modification; on a microcontroller the same policy code calls the vendor sleep interface.
- We couple the classical exponential-average idle estimator to a small set of context-selected operating modes. We show that the mode ordering follows from a monotonicity property of the energy–delay objective (Proposition 1), and we bound the resulting threshold below by the platform break-even time.
- We describe an event-driven implementation with fewer than 500 lines of code and less than 1 KB of policy state, using PowerHAL-related callbacks on Android, `epoll` with a high-resolution timer on Linux, and the firmware event loop on the IoT node.
- We evaluate the mechanism on smartphone, tablet, and IoT-node platforms with five workloads, report energy, latency, and AR/VR rendering indicators, and state explicitly which conclusions the present data do and do not support.

Section II reviews related power-management approaches and the design requirements. Section III presents the framework, including the break-even analysis and the context mechanism. Section IV describes the implementation, Section V the

G. Lim is with the Department of Electrical and Computer Engineering, Sungkyunkwan University, 2066 Seobu-ro, Jangsan-gu, Suwon, Republic of Korea (e-mail: leemgs@g.skku.edu).

methodology, and Section VI the results. Section VII discusses the results, threats to validity, and limitations, and Section VIII concludes.

II. BACKGROUND AND RELATED WORK

A. Classical Dynamic Power Management

DPM policies decide when an idle component should enter a lower-power state whose entry and exit cost time and energy [1]. A transition is profitable only if the subsequent idle period exceeds the *break-even time* T_{be} , defined in Section III-B. Timeout policies wait for a fixed period before shutting down. A timeout equal to T_{be} is 2-competitive with respect to an offline optimal policy for a two-state device [14], and the analysis extends to devices with multiple sleep states [15].

Predictive policies estimate the length of the upcoming idle period from history. Srivastava *et al.* [2] used regression on previous active and idle periods, and Hwang and Wu [3] used an exponential average of past idle intervals together with correction mechanisms for misprediction. Adaptive disk spin-down policies adjust the timeout online from the observed cost of recent decisions [4], [5]. Stochastic formulations model request arrivals and optimize expected power under a performance constraint [6].

B. Operating-System and Protocol Mechanisms

Modern kernels embed history-based idle selection. The Linux `cpuidle` framework chooses a processor idle state per CPU from recent idle history and predicted wake-up time through its governors [7], and device runtime power management offers per-device autosuspend delays. These mechanisms are effective but operate on processor idle states or individual devices, and they do not receive application context such as “an AR session is active.” Mobile platforms add system-level policies, and prior work has shown that user and network context can expose idle intervals suitable for energy saving [16], [17]. In IoT networks, energy is also managed at the protocol layer through routing objective functions [18], radio power-saving modes [8], and predictive or learned sleep schedules [9], [13].

C. Learning-Based Policies

Learning-based schedulers predict future idle intervals or workload states, including deep reinforcement learning for energy-harvesting IoT [12] and QoS-aware co-optimization of scheduling and frequency governing on mobile devices [19]. Prediction can reduce unnecessary transitions, but it introduces feature extraction, model state, inference, training or retraining, and sometimes hardware acceleration. These costs may be acceptable on high-end systems but are less attractive for small IoT nodes or products that prioritize deterministic, inspectable behavior.

D. Positioning and Design Requirements

Table I summarizes where UATimer sits. It complements rather than replaces processor-level DVFS and idle governors and radio-specific power saving. Its role is to decide, from

TABLE I
POSITIONING OF UATIMER RELATIVE TO COMMON TECHNIQUES

Technique	Primary control variable	Typical strength	Main limitation
Fixed timeout	Idle duration	Simplicity, determinism Early shutdown	No workload adaptation No application context
Predictive shutdown	Predicted idle length	Fine-grained, in-kernel	Kernel/ISA specific; no app context
OS idle governor	Per-CPU idle state	Compute efficiency	Does not decide suspend timing
DVFS/task mapping	Voltage/frequency, placement	Communication energy	Tied to protocol schedule
Protocol duty cycle	Radio/sampling schedule	Rich adaptation	Training/inference complexity
ML scheduler	Predicted idle/workload state	Portable, inspectable	Coarse, rule-based context
UATimer	Bounded adaptive threshold		

application-level activity and context, when a platform-defined low-power transition is worthwhile.

An IoT-oriented policy must operate under constraints that are less visible on server-class systems. First, policy state should remain small enough to coexist with application buffers and protocol stacks on memory-constrained nodes. Second, adaptation should not require a persistent training pipeline, because many deployed devices have intermittent connectivity and long maintenance cycles. Third, the control path should be inspectable: a developer should be able to determine why a particular timeout was selected and to impose platform-specific minimum or maximum values. Finally, the same policy should remain usable when the endpoint changes from an interactive mobile device to a sensing node. These requirements translate into five design goals:

- 1) **Low overhead:** constant-size state and $O(1)$ work per event.
- 2) **Responsiveness:** wake-up delay should remain suitable for interactive use.
- 3) **Adaptivity:** the timeout should follow changing idle behavior.
- 4) **Portability:** dependence only on generic timer and event mechanisms and a platform suspend primitive.
- 5) **Deployability:** no kernel or driver modification on Linux and Android.

III. USER-AWARE ADAPTIVE TIMER FRAMEWORK

A. System Architecture

Fig. 1 shows the architecture. The *User Interaction Monitor* captures user input and background events and supplies a context label. The *Adaptive Timer Manager* maintains the idle estimate and computes the effective threshold. The *Power Controller* invokes the platform low-power transition and resumes the system on a new event. The framework does not alter the kernel scheduler or device drivers; the Power Controller is the only platform-specific boundary.

B. Break-Even Time and Threshold Bounds

Let P_i and P_s denote the platform power in the idle-wait and low-power states, and let E_{tr} and τ_{tr} denote the total energy and time of one entry–exit transition. Following the

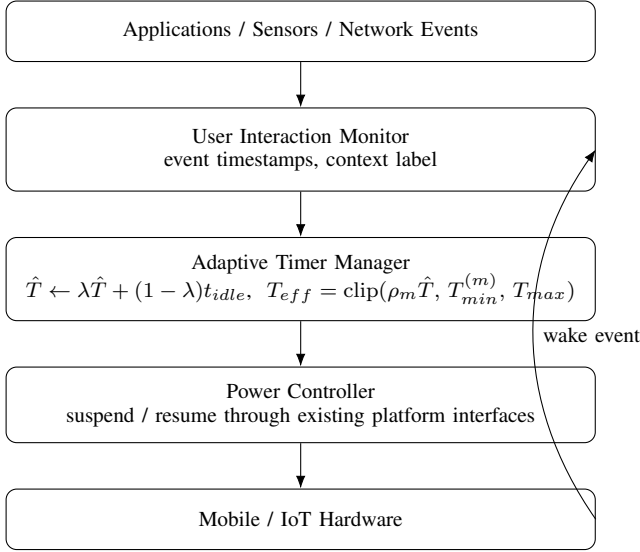


Fig. 1. Architecture of the user-aware adaptive timer framework.

standard DPM analysis [1], suspending is profitable only for idle periods longer than

$$T_{be} = \max\left(\tau_{tr}, \frac{E_{tr} - P_s \tau_{tr}}{P_i - P_s}\right). \quad (1)$$

T_{be} is a platform property that can be measured once per device and low-power state. UATimer constrains its effective threshold to

$$T_{min}^{(m)} \leq T_{eff} \leq T_{max}, \quad T_{min}^{(m)} \geq T_{be}, \quad (2)$$

where the lower bound may depend on the operating mode m (Section III-D) and T_{max} limits idle energy waste. The floor ensures that the policy never arms a threshold that would make every resulting transition unprofitable, which a pure history estimate could otherwise do when events are very frequent.

C. Adaptive Idle Estimate

When the next event occurs after an idle period, the monitor records the observed interval t_{idle} between consecutive events and updates the estimate

$$\hat{T}_{new} = \lambda \hat{T}_{old} + (1 - \lambda)t_{idle}, \quad (3)$$

where $0 \leq \lambda < 1$ is the temporal smoothing coefficient. This is the exponential-average predictor of [3]; UATimer uses it as the adaptive center of the threshold rather than as a shutdown oracle. When the device was suspended, t_{idle} includes the resume latency, which biases $\hat{T}$ upward by at most τ_{tr} .

For a stationary sequence with constant idle interval t^* , repeated application gives $\hat{T}_k - t^* = \lambda^k(\hat{T}_0 - t^*)$, so the estimate converges geometrically. More generally,

$$\hat{T}_k = \lambda^k \hat{T}_0 + (1 - \lambda) \sum_{i=0}^{k-1} \lambda^i t_{k-i}, \quad (4)$$

so recent observations dominate, and λ sets how many events are needed to follow a workload shift. After a step change in the idle interval, the residual error decays by a factor of $1/e$ after approximately $-1/\ln \lambda$ events.

TABLE II
OPERATING MODES AND ENERGY-DELAY WEIGHTS

w_E	w_D	ρ_m	Priority mode
0.8	0.2	$1 - 0.6\alpha$	Energy-conservative
0.5	0.5	1	Balanced
0.2	0.8	$1 + 0.6\alpha$	Latency-critical

D. Context-Selected Operating Modes

Observed idle duration alone cannot represent application requirements [11], [19]. We express the energy-responsiveness preference through

$$J(T) = w_E E(T) + w_D D(T), \quad (5)$$

where $E(T)$ is the energy and $D(T)$ the mean response delay under threshold T , and $w_E, w_D \geq 0$ are policy weights distinct from λ . Both terms are normalized by their values under the platform baseline so that the weights are dimensionless. UATimer does not minimize J online, because $E(T)$ and $D(T)$ are not known in closed form at run time. Instead, the objective determines the *direction* in which context should move the threshold.

Proposition 1: If $D(T)$ is nonincreasing in T , the set of minimizers of $J(T)$ is nondecreasing in the ratio $r = w_D/w_E$ in the strong set order.

Proof: Write $\tilde{J}(T; r) = E(T) + rD(T)$, which has the same minimizers as J . For $T' > T$ and $r' > r$, $[\tilde{J}(T'; r') - \tilde{J}(T; r')] - [\tilde{J}(T'; r) - \tilde{J}(T; r)] = (r' - r)(D(T') - D(T)) \leq 0$. Hence $\tilde{J}$ has decreasing differences in (T, r) , and the claim follows from Topkis' monotonicity theorem. ■

A longer threshold means fewer suspensions and therefore fewer wake-up penalties, so the assumption on D holds for the timeout policies considered here. Proposition 1 states that a mode that weights delay more heavily should never select a *shorter* threshold than a mode that weights energy more heavily. UATimer implements this ordering with three fixed modes (Table II) and a mode-dependent scale

$$T_{eff} = \min(T_{max}, \max(T_{min}^{(m)}, \rho_m \hat{T})), \quad (6)$$

$$\rho_m = 1 + \alpha (w_D^{(m)} - w_E^{(m)}), \quad (7)$$

with $0 \leq \alpha < 1$, so that $\rho_m > 0$ and ρ_m increases with $w_D^{(m)}$. The weights are looked up per mode rather than incremented per event, so they remain in $[0, 1]$ and sum to one. In addition, the latency-critical mode may set an *inhibit* flag while the application reports an active interactive session, which suppresses suspension regardless of $\hat{T}$. The floor in Eq. (2) and the inhibit flag address a failure mode of pure history tracking: under 60-Hz rendering, inter-event intervals of about 16.7 ms would otherwise drive $\hat{T}$ toward the frame period and produce the most aggressive suspension exactly when it is least acceptable.

The context label is produced by a rule table keyed on the application class of the foreground or registering application and on an optional time-of-day window. Examples are AR/VR→latency-critical and night-time sensing→energy-conservative [confirm rule table and mode assigned to each evaluated workload]. The mechanism is deliberately coarse; it

Algorithm 1 UATimer event-driven control

Require: λ, α, T_{max} , per-mode $(w_E^{(m)}, w_D^{(m)}, T_{min}^{(m)})$

```

1:  $\hat{T} \leftarrow \hat{T}_0; t_{last} \leftarrow \text{now}(); m \leftarrow \text{Balanced}$ 
2: procedure ONEVENT
3:    $t \leftarrow \text{now}(); t_{idle} \leftarrow t - t_{last}; t_{last} \leftarrow t$ 
4:    $\hat{T} \leftarrow \lambda \hat{T} + (1 - \lambda) t_{idle}$  ▷ Eq. (3)
5:   REARM
6: end procedure
7: procedure ONCONTEXTCHANGE( $c$ )
8:    $m \leftarrow \text{Mode}(c)$  ▷ rule-table lookup
9:   REARM
10: end procedure
11: procedure REARM
12:    $\rho \leftarrow 1 + \alpha (w_D^{(m)} - w_E^{(m)})$ 
13:    $T_{eff} \leftarrow \min(T_{max}, \max(T_{min}^{(m)}, \rho \hat{T}))$  ▷ Eq. (6)
14:   if inhibit( $m$ ) then
15:     cancel timer
16:   else
17:     arm one-shot timer at  $t_{last} + T_{eff}$ 
18:   end if
19: end procedure
20: procedure ONTIMEREXPIRY
21:   if no pending work and not inhibit( $m$ ) then
22:     SUSPEND ▷ platform primitive
23:   end if
24: end procedure
```

trades the ability to learn latent context for a mapping that a developer can audit.

E. Algorithm and Event Semantics

Algorithm 1 gives the control flow. After an event is serviced, the system enters the idle-wait state and arms a one-shot timer at T_{eff} . A new event arriving before expiry cancels the timer and produces a new idle observation. If the timer expires, the Power Controller requests a low-power state, and the next eligible user, sensor, or network event resumes the device and closes the current idle sample. The policy is therefore event-driven: it performs no polling, and each event costs a constant number of scalar operations. This formulation separates *policy* (how T_{eff} is selected) from *mechanism* (how the platform enters or exits low power).

F. Interaction With Periodic Workloads

Equation (6) clarifies a property that a pure mean-tracking timeout would hide. For a strictly periodic event stream with period P , $\hat{T} \rightarrow P$. In the balanced mode ($\rho_m = 1$), the timer then expires close to the next event, and few profitable suspensions occur. In the energy-conservative mode ($\rho_m < 1$), the timer expires after $\rho_m P$ and the device sleeps for approximately $(1 - \rho_m)P$ per period, which is profitable when $(1 - \rho_m)P > T_{be}$. Periodic sensing workloads should therefore be assigned the energy-conservative mode, and the achievable saving grows with the gap between the sampling period and T_{be} .

G. Parameterization

Configuration proceeds in two stages. The platform first measures T_{be} and selects $T_{min}^{(m)} \geq T_{be}$ and T_{max} from its

TABLE III
POLICY PARAMETERS USED IN THE EVALUATION

Symbol	Meaning	Value
λ	Temporal smoothing coefficient	[value]
α	Mode scaling strength	[value]
$\hat{T}_0$	Initial idle estimate	[mobile / IoT]
T_{be}	Break-even time, Eq. (1)	[per platform]
$T_{min}^{(m)}$	Mode-dependent floor	[per mode]
T_{max}	Upper bound	[mobile / IoT]

TABLE IV
QUALITATIVE POLICY CHARACTERISTICS

Property	Baseline	Fixed	LR predictor	UATimer
Online adaptation	OS dep.	No	Inputs only ^a	Yes
Training required	–	No	Offline fit	No
Context input	OS dep.	No	No	Yes
Break-even floor	OS dep.	No	No	Yes
Per-event cost	OS dep.	$O(1)$	$O(1)$ inference	$O(1)$
Kernel change	No	No	No	No
Policy state	OS dep.	1 scalar	4 coeff. (<64 B)	<1 KB

^aPredictions use recent idle intervals, but the coefficients are fixed after offline fitting.

wake-up characteristics. The application then supplies context, which selects the mode. Within those limits, Eq. (3) adapts continuously. Table III lists the parameters and the values used in the experiments. None of them requires a learned model. If a future implementation introduces TinyML or statistical prediction, the predictor can replace $\hat{T}$ while retaining the same bounded interface.

H. Policy Complexity

Table IV summarizes the qualitative differences among the evaluated policies, based on the implemented mechanisms rather than on additional measurements. The per-policy scalar state of UATimer ($\hat{T}$, t_{last} , the current mode, three weight pairs, and bounds) amounts to tens of bytes. The 1 KB bound also covers the context rule table.

IV. IMPLEMENTATION

A. Platform Integration

The policy logic (Algorithm 1) is written once and linked against a small platform adaptation layer that provides event notification, a one-shot timer, and the suspend primitive. On Android, a user-space daemon registers PowerHAL-related callbacks to observe relevant activity and coordinate low-power transitions. On Linux, it uses `epoll`-based input hooks to detect events and a high-resolution timer to arm and disarm the threshold. On these platforms, deployment changes are limited to user-space startup and manifest/init integration rather than kernel or driver patches.

The Cortex-M4 IoT node has no user/kernel separation. There, the same policy code is compiled into the firmware as an application-level module that is invoked from the firmware event loop, and SUSPEND maps to the MCU's low-power mode [name RTOS or bare-metal environment, sleep mode, and wake sources]. The “no kernel modification” property therefore applies to the Linux and Android platforms. On the

node, the corresponding property is that the vendor power-management code is not changed.

The implementation is below 500 lines of code, and the persistent policy state is below 1 KB. These properties reduce the maintenance burden and avoid a model-management pipeline.

B. Cross-Architecture Portability

The control logic depends on timer and event interfaces rather than processor-specific instructions. Besides the three evaluated platforms, the policy module was additionally exercised on an Intel x86 Linux laptop and an ESP32-class RISC-V board [state what was run on the RISC-V board: firmware build, functional test, or trace replay]. This is a portability check, not a cross-architecture performance evaluation; the present dataset contains no energy or latency measurements for these two systems.

C. Lightweight Learning Baseline

For comparison, a linear-regression (LR) scheduler predicts the next idle interval from the previous three idle intervals and a bias term. The model is fitted offline by least squares on 10^3 idle-interval samples [state the workload(s) the samples came from and whether one model was fitted per workload] and stores four scalar coefficients, i.e., less than 64 B of model state. The reported per-event inference time is below 0.3 ms on a Cortex-A53 platform. This baseline represents a deliberately lightweight learning approach and belongs to the same family of linear idle predictors as [2]. It is not a state-of-the-art neural or TinyML predictor, so the comparison supports only the claim that UATimer reaches a similar operating point to this specific predictor without offline fitting.

V. EXPERIMENTAL METHODOLOGY

A. Platforms and Measurement

The evaluation covers three representative device classes:

- an Android smartphone based on a Qualcomm Snapdragon platform with a 4000 mAh battery [SoC model, Android and kernel versions],
- a tablet based on an ARM Cortex-A73 platform with a 6000 mAh battery [SoC model, OS version], and
- a Cortex-M4 IoT sensor node powered by a coin-cell battery [MCU model, radio].

Energy on the mobile devices was measured using a Monsoon power monitor, and an external meter was used for the IoT node [meter model and sampling rate]. Energy is integrated over a fixed measurement window per run [window length per workload]. Event latency was obtained from timestamp logging as the time from event arrival to the start of its handling. Wi-Fi was enabled and display brightness was fixed at 50% for the mobile-device experiments.

B. Workloads

Five workload classes were used [state which device each workload ran on]:

- **Web browsing:** intermittent touch input mixed with background network activity;

- **Video playback:** continuous decoding with relatively little user input;
- **AI inference:** long-running on-device DNN execution;
- **AR/VR gaming:** 60-Hz rendering with frequent sensor and interaction events; and
- **IoT sensing:** periodic sensor sampling and wireless transmission.

For the video and AI workloads, the low-power transitions managed by UATimer are those available while the main pipeline continues to run [specify the managed state: e.g., display, radio, peripheral runtime PM, or system suspend between inference batches]. System-wide suspend during active decoding is not implied.

C. Compared Policies

Four policies were compared:

- 1) **Baseline:** default platform behavior without additional timeout tuning [define: e.g., stock Android idle/Doze settings; default firmware behavior on the node];
- 2) **Fixed timeout:** a static timeout of 3 s for mobile workloads and 5 s for the IoT workload;
- 3) **LR predictor:** the lightweight linear-regression scheduler described above; and
- 4) **UATimer:** the proposed policy with the parameters in Table III.

The fixed timeouts are representative settings and were not tuned to each platform's T_{be} ; a break-even-tuned timeout [14] would be a stronger static baseline. Each experiment was repeated five times, and Table V reports the arithmetic means. The archived dataset used to prepare this manuscript does not include the per-run samples. For that reason, this version reports no standard deviations, confidence intervals, or significance tests, and it draws no conclusion from differences whose size is comparable to typical run-to-run variation.

VI. RESULTS

A. Energy and Latency

Table V summarizes the measurements, and Table VI reports reductions computed directly from them; these percentages are arithmetic transformations, not additional runs. Relative to the default baseline, UATimer reduces energy by 20.5–37.5% (mean 27.7% across the five workloads). Relative to the fixed-timeout policy, the reduction is 7.9–16.7%. UATimer has the lowest mean energy in every workload, and its mean event delay remains below 50 ms throughout.

Fig. 2 plots each policy as a point in the energy–delay plane, with energy normalized to the baseline of the same workload. Power management is a multi-objective problem. The baseline has the lowest delay and the highest energy, and the fixed timeout reduces energy at the largest delay cost. UATimer lies on the lower-left side of the fixed-timeout and LR points for every workload. Relative to the baseline, however, UATimer increases mean delay by 5–15 ms (up to 50% for web browsing and 60% for video playback). The appropriate claim is therefore an improved energy–delay trade-off, not unchanged responsiveness.

TABLE V
EVALUATION RESULTS ACROSS WORKLOADS (MEANS OF FIVE RUNS)

Workload	Policy	Energy (mJ)	Delay (ms)	Frame drop (%)	Jitter (ms)
Web Browsing	Baseline	1200	30	–	–
	Fixed timeout	900	50	–	–
	LR predictor	780	47	–	–
	UATimer	750	45	–	–
Video Playback	Baseline	1800	25	–	–
	Fixed timeout	1500	42	–	–
	LR predictor	1380	41	–	–
	UATimer	1350	40	–	–
AI Inference	Baseline	2200	28	–	–
	Fixed timeout	1900	40	–	–
	LR predictor	1780	39	–	–
	UATimer	1750	38	–	–
AR/VR Gaming (60 Hz)	Baseline	2500	20	2.5	3.0
	Fixed timeout	2100	30	3.2	3.5
	LR predictor	1850	29	1.8	2.7
	UATimer	1800	28	0.5	1.2
IoT Sensing	Baseline	400	12	–	–
	Fixed timeout	320	18	–	–
	LR predictor	300	17	–	–
	UATimer	290	17	–	–

TABLE VI
ENERGY REDUCTION COMPUTED FROM TABLE V

Workload	vs. Baseline (%)	vs. Fixed (%)	vs. LR (%)
Web browsing	37.5	16.7	3.8
Video playback	25.0	10.0	2.2
AI inference	20.5	7.9	1.7
AR/VR gaming	28.0	14.3	2.7
IoT sensing	27.5	9.4	3.3
Mean	27.7	11.6	2.7

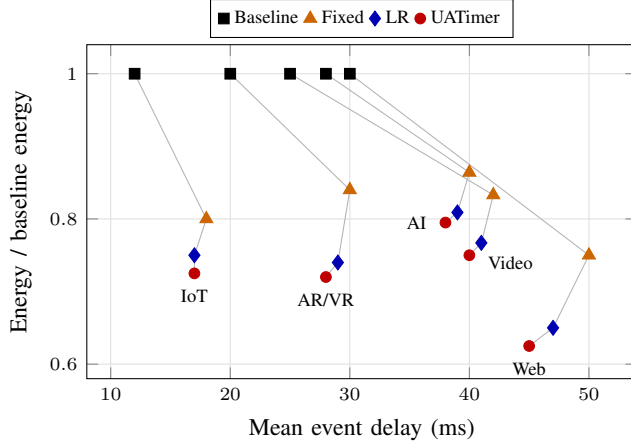


Fig. 2. Energy–delay operating points per workload (energy normalized to the baseline of the same workload; lines connect the policies of one workload). Lower-left is better.

B. AR/VR Rendering Quality

The AR/VR workload shows why aggressive energy saving must be bounded. The fixed-timeout policy reduces energy but increases frame drops from 2.5% to 3.2% and jitter from 3.0 ms to 3.5 ms. With the latency-critical mode, UATimer

reduces energy from 2500 mJ to 1800 mJ while reporting 0.5% frame drops and 1.2 ms jitter. The comparison with the fixed timeout is consistent with the mechanism: the mode floor and inhibit flag avoid transitions during active rendering, which a fixed 3 s timeout cannot distinguish.

UATimer also reports lower frame drops and jitter than the baseline, which performs no additional suspension. The timeout mechanism alone does not explain this observation. Candidate explanations, such as reduced thermal throttling from lower average power or changed background scheduling, were not measured. We therefore claim only that the latency-critical mode did not degrade rendering quality relative to the other policies, not that it improves rendering quality. The mean event delay of 28 ms, which exceeds the 16.7 ms frame period, refers to input and sensor events handled by the policy monitor rather than to frame presentation latency.

C. Comparison With the Lightweight LR Predictor

UATimer’s mean energy is 1.7–3.8% below that of the LR predictor, with similar mean delay. Without per-run dispersion data, differences of this size cannot be distinguished from run-to-run variation. We therefore do not claim that UATimer outperforms the predictor. We claim that it reaches a comparable operating point without offline fitting, and that it adds context input and a break-even floor, which the predictor lacks. This is consistent with the shared estimator family: both policies are linear filters over recent idle intervals, so large differences in their idle predictions are not expected on these workloads.

VII. DISCUSSION

A. Why the Method Helps and Workload Dependence

A fixed timeout assumes a stationary environment, whereas interactive systems are non-stationary. Web activity alternates between bursts and pauses, sensing workloads can become

sparse, and latency-sensitive applications can temporarily dominate the system. Equation (3) lets the threshold follow these changes, and Eq. (6) shifts it in the direction prescribed by Proposition 1 when the context changes. The magnitude of the gain depends on the availability of idle intervals longer than T_{be} , which is consistent with prior observations that the energy impact of power-state decisions is highly workload- and interface-dependent [8], [10]. Web browsing contains many exploitable gaps, whereas the continuous pipelines of video decoding and AI inference provide fewer. Once the available gaps are captured, additional prediction complexity has limited room to improve total energy, which explains the narrow margin to the LR predictor.

B. Failure Modes and Safe Integration

Suspending can be undesirable even after a long idle interval. Examples include non-deferrable background operations, protocol-critical communication windows, and device-specific transitions with high wake cost. The architecture handles these cases at the integration boundary: platform or application context can inhibit a transition, select the latency-critical mode, or tighten the bounds in Eq. (2). UATimer is thus a policy layer that complements existing platform safety and power-management controls. On Linux and Android, the user-space placement also limits the failure domain. If the daemon is disabled or cannot obtain its event hooks, the underlying OS policy remains in effect, so the adaptive component can be enabled per product or workload. The trade-off is that user-space observation may not expose every device-level event.

C. Threats to Validity

Internal validity depends on stable measurement conditions. Wi-Fi state and display brightness were fixed, and each experiment was repeated five times, but uncontrolled background activity can still affect mobile energy measurements, and per-run dispersion is not available. *Construct validity* is limited by the chosen metrics. Total energy and mean event delay capture the primary trade-off, but tail latency, the number of suspend/resume transitions, state residency, the overhead of the policy daemon itself, and battery-lifetime projections are not reported. *External validity* is limited by the three evaluated platforms, of which only one is an IoT node, and by the absence of radio-specific experiments.

D. Limitations and Evidence Boundaries

The present data support four conclusions. First, UATimer has the lowest mean energy of the four evaluated policies in each of the five workloads. Second, its mean event delay remains below 50 ms, although it is 5–15 ms higher than the baseline. Third, in the AR/VR workload its frame-drop and jitter values are not worse than those of the comparison policies. Fourth, the policy is a small component that requires no kernel modification on Linux and Android.

The data do *not* establish statistical significance, superiority over the LR predictor or over modern TinyML and reinforcement-learning policies, radio-specific IoT energy behavior, battery-lifetime gains, or cross-architecture energy

gains. The context rules are manually defined, the fixed-timeout baseline was not tuned to T_{be} , the overhead of the policy itself was not measured, and the lower AR/VR frame-drop rate relative to the baseline remains unexplained. The highest-priority follow-up measurements are therefore: (i) at least ten independent repetitions per condition with dispersion and significance tests; (ii) ablations separating Eq. (3), the mode scaling, and the floor, together with sensitivity to λ and α ; (iii) break-even-tuned timeout, Linux `cpuidle`, and TinyML baselines; (iv) additional MCU platforms with BLE, LoRa, or NB-IoT radios and battery-lifetime estimates; and (v) tail latency, transition counts, and policy overhead.

E. Reproducibility

A reference implementation of the policy core (Algorithm 1 and Eqs. (3), (6), (7)), the Linux `epoll/timerfd` adaptation layer, the linear-regression baseline of Section IV-C, unit tests for the stated properties, and a trace-driven functional harness are archived at <https://github.com/leemgs/uatimer>. The harness exercises the control logic on event traces and reports transition behavior; it does not model device power and does not reproduce the measured energy values. The per-platform tuned parameters, the recorded workload traces, and the raw per-run energy samples **[add the measured parameters, event traces, and measurement scripts to the repository before submission]**. Reproducibility is particularly important for lightweight power-management mechanisms, because small changes in timeout parameters, device state, and workload timing can change the measured savings.

VIII. CONCLUSION

This paper presented UATimer, a user-aware adaptive timer for energy-efficient and responsive power management in heterogeneous IoT and mobile-edge systems. UATimer reuses the classical exponential-average idle estimator and bounds it below by the platform break-even time. It shifts the threshold through context-selected modes whose ordering follows from the energy–delay objective, and it separates this policy from the platform suspend mechanism so that the same code runs in user space on Linux and Android and in firmware on a microcontroller. The implementation requires fewer than 500 lines of code and less than 1 KB of state. Across the reported browsing, video, AI, AR/VR, and sensing measurements, UATimer reduces mean energy by 20.5–37.5% relative to the default baseline and by 7.9–16.7% relative to a fixed timeout, keeps mean event latency below 50 ms, and reaches energy values comparable to a lightweight LR predictor without offline fitting. Future work will address the measurements listed in Section VII-D and extend the policy to the joint management of CPU, GPU, display, and radio states.

ACKNOWLEDGMENT

OpenAI ChatGPT and Anthropic Claude were used to assist with drafting, restructuring, and revising portions of the Introduction, Background and Related Work, the formal description in Section III, the Discussion, and language throughout the manuscript. The author reviewed and edited the generated text

and takes responsibility for the final content. The AI systems were not used to generate, alter, or fabricate experimental measurements or reported results.

REFERENCES

- [1] L. Benini, A. Bogliolo, and G. De Micheli, "A survey of design techniques for system-level dynamic power management," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 8, no. 3, pp. 299–316, Jun. 2000.
- [2] M. B. Srivastava, A. P. Chandrakasan, and R. W. Brodersen, "Predictive system shutdown and other architectural techniques for energy efficient programmable computation," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 4, no. 1, pp. 42–55, Mar. 1996.
- [3] C.-H. Hwang and A. C.-H. Wu, "A predictive system shutdown method for energy saving of event-driven computation," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 5, no. 2, pp. 226–241, Apr. 2000.
- [4] F. Douglass, P. Krishnan, and B. N. Bershad, "Adaptive disk spin-down policies for mobile computers," in *Proc. 2nd USENIX Symp. Mobile and Location-Independent Computing*, 1995.
- [5] D. P. Helmbold, D. D. E. Long, and B. Sherrod, "A dynamic disk spin-down technique for mobile computing," in *Proc. 2nd Annu. Int. Conf. Mobile Computing and Networking (MobiCom)*, 1996.
- [6] T. Simunic, L. Benini, P. Glynn, and G. De Micheli, "Event-driven power management," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 20, no. 7, pp. 840–857, Jul. 2001.
- [7] V. Pallipadi, S. Li, and A. Belay, "cpuidle: Do nothing, efficiently," in *Proc. Linux Symp.*, Ottawa, ON, Canada, 2007.
- [8] F. Michelinakis, A. S. Al-Selwi, M. Capuzzo, A. Zanella, K. Mahmood, and A. Elmokashfi, "Dissecting energy consumption of NB-IoT devices empirically," *IEEE Internet Things J.*, vol. 8, no. 2, pp. 1224–1242, Jan. 2021, doi: 10.1109/JIOT.2020.3013949.
- [9] J. Sheth, C. Miremadi, A. Dezfouli, and B. Dezfouli, "EAPS: Edge-assisted predictive sleep scheduling for 802.11 IoT stations," *IEEE Syst. J.*, vol. 16, no. 1, pp. 591–602, Mar. 2022, doi: 10.1109/JSYST.2021.3063922.
- [10] A. Carroll and G. Heiser, "An analysis of power consumption in a smartphone," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2010.
- [11] J. Lee, K. Lee, E. Jeong, J. Jo, and N. B. Shroff, "CAS: Context-aware background application scheduling in interactive mobile systems," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 5, pp. 1013–1029, May 2017, doi: 10.1109/JSAC.2017.2676918.
- [12] H. Ko, H. Choi, and S. Pack, "Restoration-aware sleep scheduling framework in energy harvesting Internet of Things: A deep reinforcement learning approach," *IEEE Trans. Sustainable Comput.*, vol. 10, no. 1, pp. 190–198, Jan./Feb. 2025, doi: 10.1109/TSUSC.2024.3442918.
- [13] D. E. Ruiz-Guirola, S. Montejó-Sánchez, I. Leyva-Mayorga, Z. Han, P. Popovski, and O. L. A. Lopez, "Energy management and wakeup for IoT networks powered by energy harvesting," *IEEE Internet Things J.*, vol. 13, no. 9, pp. 18574–18591, May 2026, doi: 10.1109/JIOT.2026.3664201.
- [14] A. R. Karlin, M. S. Manasse, L. A. McGeoch, and S. Owicki, "Competitive randomized algorithms for nonuniform problems," *Algorithmica*, vol. 11, no. 6, pp. 542–571, Jun. 1994.
- [15] S. Irani, S. Shukla, and R. Gupta, "Online strategies for dynamic power management in systems with multiple power-saving states," *ACM Trans. Embedded Comput. Syst.*, vol. 2, no. 3, pp. 325–346, Aug. 2003.
- [16] I. Pefkianakis, J. Chandrashekar, and H. Lundgren, "User-driven idle energy save for 802.11x mobile devices," in *Proc. IEEE Int. Conf. Mobile Ad Hoc Sensor Syst. (MASS)*, 2014, pp. 282–290, doi: 10.1109/MASS.2014.50.
- [17] Y. Kim and J. Kim, "Personalized Diapause: Reducing radio energy consumption of smartphones by network-context aware dormancy predictions," in *Proc. Workshop Power-Aware Computing and Systems (HotPower)*, 2012.
- [18] B. Safaei, A. M. Hosseini Monazzah, and A. Ejlali, "ELITE: An elaborated cross-layer RPL objective function to achieve energy efficiency in Internet-of-Things devices," *IEEE Internet Things J.*, vol. 8, no. 2, pp. 1169–1182, Jan. 2021, doi: 10.1109/JIOT.2020.3011968.
- [19] Q. Sang, J. Yan, R. Xie, C. Hu, K. Suo, and D. Cheng, "QoS-aware power management via scheduling and governing co-optimization on mobile devices," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 13654–13669, Dec. 2024, doi: 10.1109/TMC.2024.3438267.